

T3 Racing Line — Code Guide

Every file, class and important function of `t3_racing_line/`, with the numbers from the test runs.

Companion to `task3.md` (the T3 spec) and `docs/T1_CODE_GUIDE.pdf`. State: steps 0–6 of the spec built, 27 tests passing, waiting for the real T2.

0. The big picture

T3 = Racing line (minimum curvature). T1 gives the track (a centerline + how far each wall is). T3 finds the line the car should drive: the one that **bends as little as possible** while staying inside the walls with a safety margin. Later, T4 puts a speed on every point of that line and the controller follows it.

Input

A track model (from T2; for now our stand-in built from T1's `track.csv`): at any distance s it gives x , y , heading, κ , w_{left} , w_{right} .
Plus the car's numbers: width, safety buffer, max steering, wheelbase.

Output

`racing_line.csv` (s , x , y , heading, κ , w_{left} , w_{right} , a row every ~ 0.10 m) +
`racing_line_meta.json` (how it was made, did it converge). Loadable back as a track model.

One row of `racing_line.csv` (T1 synthetic shape)

```
s,      x,      y,      heading, kappa, w_left, w_right
0.000, 6.836, -0.267, 1.115,   0.395, 1.499,  0.201
```

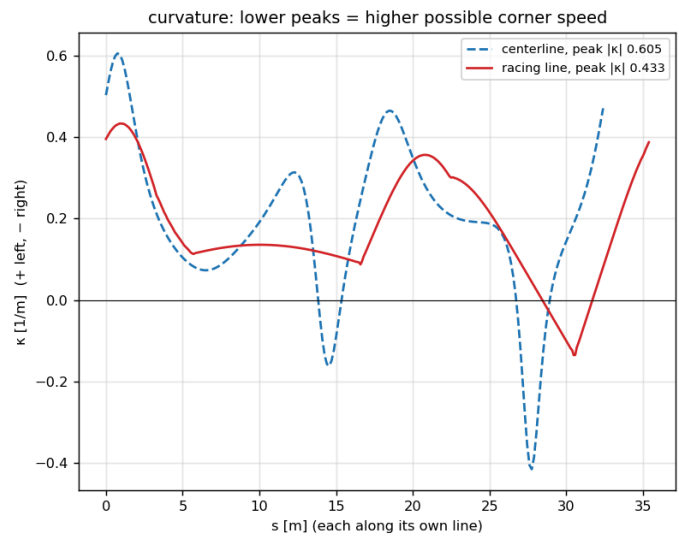
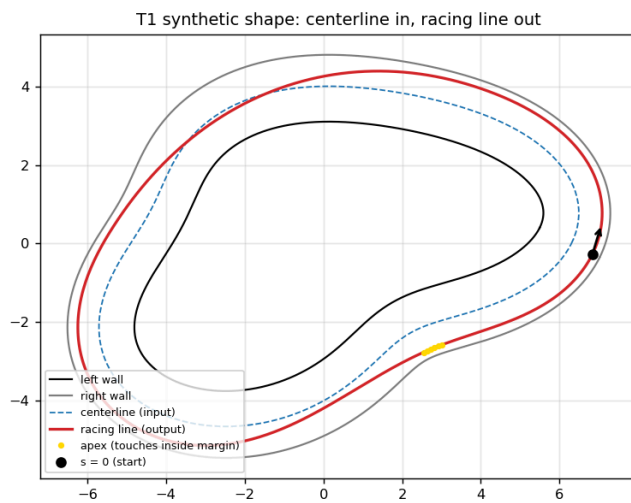
"At the start line, the racing line is at (6.84, -0.27), pointing 1.115 rad (64°) from the +x axis, **turning left** with $\kappa = 0.395$ 1/m (a circle of radius $1/0.395 = 2.5$ m). The left wall is 1.50 m away, the right wall 0.20 m away" — the line hugs the outside wall at exactly the margin, the way a race car enters a corner wide.

Why "minimum curvature"?

In a corner the tyres can only push sideways so hard: $v_{\text{max}} = \sqrt{a_{\text{lat_max}} / |\kappa|}$. Smaller curvature → faster possible corner speed. So we minimise $\sum \kappa^2$ over the whole lap: squaring treats left and right turns the same, and punishes one sharp bend more than several gentle ones.

The key idea in four steps

1. Put stations every ~ 15 cm on the centerline: r_i , left normal n_i
2. Let the line move sideways at each station: $p_i = r_i + \alpha_i \cdot n_i$ ($\alpha > 0 = \text{left}$)
but not closer than the margin to a wall: $\alpha_{\text{min}_i} \leq \alpha_i \leq \alpha_{\text{max}_i}$
3. Curvature of the new line is $\sim \text{LINEAR}$ in α : $\kappa \approx \kappa_{\text{ref}} + M \alpha$
→ minimise $\|\kappa_{\text{ref}} + M \alpha\|^2$ = a Quadratic Program (QP), solved exactly by OSQP
4. "Almost" linear → repeat around the new line until it stops moving (< 1 cm)



Left: the input centerline (dashed) and the racing line T3 produces (red). It swings to the outside of the big left-hand loop and cuts the right-hand dent to the inside (yellow = apex). Right: curvature along each line — the peak drops from 0.605 to 0.433 1/m.

1. The pipeline and the files

```
track.csv (T1)    32.56 m centerline, point every 5 cm
|
▼ track_model_stub.py    TrackModel: spline, true arc length s, heading,  $\kappa$     (TEMPORARY T2)
|
▼ stations.py            make_stations: 217 stations, ds = 0.1500 m,
|                        normals, bounds  $\alpha_{\min}$  /  $\alpha_{\max}$  (margin 0.20 m)
|                        build_walls + widths_from_walls (ray-casting)
|
▼ min_curvature.py       build_curvature_model     $\kappa \approx \kappa_{\text{ref}} + M \alpha$ 
|                        solve_min_curvature_qp OSQP  $\rightarrow \alpha$ 
|                        iterate_min_curvature    4 rounds,
|                        max $|\alpha|$  0.600  $\rightarrow$  0.135  $\rightarrow$  0.011  $\rightarrow$  0.003 m
|
▼ export.py              build_racing_line: s = 0 on the start line,
|                        355 points every 0.0999 m,
|                        racing line 35.49 m long, widths re-measured
|                        write_racing_line  $\rightarrow$  racing_line.csv + racing_line_meta.json
|                        load_racing_line  $\rightarrow$  back to a TrackModel
```

File	Job	Main things inside	Lines
track_model_stub.py	stand-in for T2: points $\rightarrow$ smooth curve you can sample at any s	TrackModel (<code>__init__</code> , <code>from_csv</code> , <code>sample</code>), <code>_closed_cumlen</code>	76
stations.py	stations + limits: where the line may go	Stations, <code>vehicle_margin</code> , <code>make_stations</code> , <code>build_walls</code> , <code>widths_from_walls</code> , <code>_ray_distance</code>	143
min_curvature.py	the optimisation: model, cost, QP, loop	<code>kappa_max_from_steering</code> , <code>build_curvature_model</code> , <code>difference_matrix</code> , <code>curvature_cost</code> , <code>solve_min_curvature_qp</code> , <code>iterate_min_curvature</code> (+ <code>QPResult</code> , <code>IterationLog</code> , <code>IterateResult</code>)	233
export.py	final line $\rightarrow$ files $\rightarrow$ back	<code>RacingLine</code> , <code>_start_on_line</code> , <code>build_racing_line</code> , <code>build_meta</code> , <code>write_racing_line</code> , <code>read_racing_line</code> , <code>load_racing_line</code>	143

Tests (automatic, `tests/racing_line/`): one test file per code file, 27 tests. External library for the QP: `osqp` (1.1.3). Everything else: `numpy`, `scipy`.

Coming change (control team's rulebook): the *APEX Planning–Control Interface Rulebook v1.0* wants the output indexed by the **centerline** `s_track_m` with `e_y_ref_m`, `e_psi_ref_rad`, `kappa_ref_1pm`, `v_ref_mps`, at $\leq 0.05\text{--}0.10$ m, plus a manifest and a validation report. Only `export.py` will change; steps 0–5 stay. Waiting for the control team's answers.

2. track_model_stub.py — the temporary T2

Input

closed-loop points $x, y + w_{\text{left}}, w_{\text{right}}$
(first point NOT repeated), e.g. from T1's
track.csv via `TrackModel.from_csv(path)`

Output

an object with `.length` (lap length L) and
`.sample(s) → (s, x, y, heading, kappa,`
`w_left, w_right)` at any s

Why it exists: T3 needs heading and curvature, which T1 does not give (that is T2's job). This file is the smallest thing that provides the API T3 expects from T2. When the real T2 arrives, only the imports of this file change (in `min_curvature.py` and `export.py`).

`__init__` — build the curve (steps 0–3)

0) optional smoothing	<code>if smooth_sigma > 0:</code> <code>gaussian_filter1d(pts, sigma/mean_gap, mode="wrap")</code>
1) spline through the points	<code>u = chord lengths (straight distances between points)</code> <code>CubicSpline(u, points + first point again, bc_type="periodic")</code>
2) true arc length	<code>sample the spline 10× denser, add up the tiny</code> <code>segments → table $s(u)$</code> <code>self.length = last value of that table</code>
3) widths	<code>stored against true s at the original points</code>

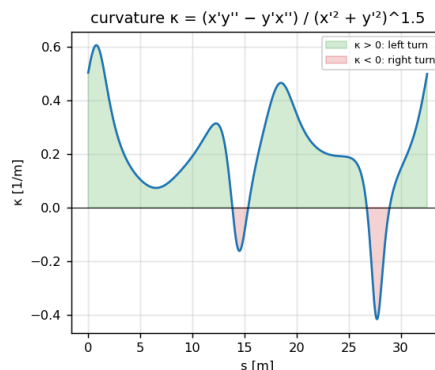
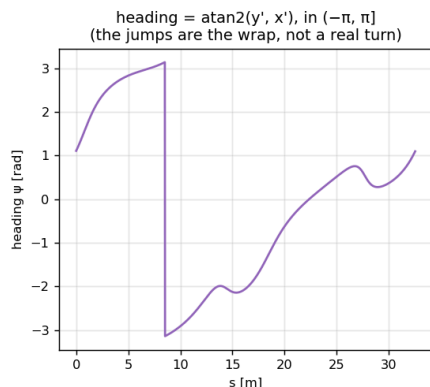
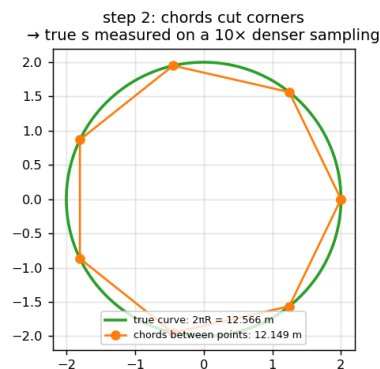
Periodic spline = the end joins the start smoothly (no kink at the start/finish line). The first point is appended once only because `CubicSpline` needs the value at $u = L$ explicitly; it is the same point.

Why step 2: the spline's own parameter u is the *chord* length, which cuts corners. A 7-point circle of radius 2 m: chords add up to 12.000 m, the real circumference is 12.566 m. With points 5 cm apart the difference is tiny, but the spec requires s to be true metres (T4's speed profile integrates over s), so we measure it.

`sample(s)` — everything at a distance s

<code>s = np.mod(s, L)</code>	<code># wrap: $s = L + 1 → 1$ m into the next lap</code>
<code>u = np.interp(s, self._s, self._u)</code>	<code># true $s →$ spline parameter u</code>
<code>p, d1, d2 = spline(u), spline(u, 1), spline(u, 2)</code>	<code># position, 1st and 2nd derivative</code>
<code>heading = arctan2(dy, dx)</code>	<code># tangent direction, 0 = +x, CCW +</code>
<code>kappa = (dx*ddy - dy*ddx) / (dx² + dy²)^{1.5}</code>	<code># signed curvature, + = left turn</code>
<code>w_left = np.interp(s, s_pts, w_left, period=L)</code>	<code># widths: linear between points</code>

Why no conversion of the derivatives: the curvature formula gives the same number whatever the curve's parameter is (u or s): it measures a shape, not a speed. That is why `sample` can differentiate by u and still return κ per metre.



1 chords are shorter than the arc → measure true s. 2 heading along the lap: the jumps are the wrap at $\pm\pi$, not a real turn. 3 curvature: positive = left turn (green), negative = right turn (red).

Worked example (test)	Expected	Checked by
circle R = 5 m, 600 points, counter-clockwise	$L = 2\pi \cdot 5 = 31.416$ m, $\kappa = +1/5 = 0.2$ everywhere	<code>test_length_is_circumference</code> , <code>test_curvature_sign_and_value</code>
same circle clockwise	$\kappa = -0.2$ (right turn)	<code>test_curvature_sign_and_value</code>
sample at s = 3.0, 3.1, 3.2	points exactly 0.1 m apart	<code>test_s_is_true_arc_length</code>
<code>sample(0)</code> vs <code>sample(L)</code>	identical	<code>test_closed_and_periodic</code>
point + $w_{\text{left}} \cdot n$ (n = $(-\sin \psi, \cos \psi)$)	on the inner wall, radius 5 - 1 = 4 m	<code>test_left_normal_points_to_left_wall</code>

Limitation: without smoothing the spline passes exactly through every point, so point noise becomes curvature noise. On Monza it gives peak $\kappa = 1.42\text{--}1.49$ 1/m (radius 0.7 m) at the chicane — partly noise. The real T2 must smooth properly (section 7).

3. stations.py — stations and limits

Input

a track model + `spacing_target` (0.15 m) +
`margin` (0.20 m) + optionally the fixed `walls`

Output

`Stations` : one value per station of everything
the optimiser needs, including the allowed
range of α

The Stations box (@dataclass)

Field	Shape	Meaning
<code>s</code>	(N,)	distance of each station, 0 ... L - ds (no station at L: same place as 0)
<code>xy</code>	(N, 2)	reference points r_i
<code>heading</code> , <code>kappa</code>	(N,)	ψ_i and κ_i of the reference line (from the track model)
<code>normals</code>	(N, 2)	left unit normals $n_i = (-\sin \psi_i, \cos \psi_i)$
<code>w_left</code> , <code>w_right</code>	(N,)	distance to each wall
<code>alpha_min</code> , <code>alpha_max</code>	(N,)	allowed sideways offset (right limit, left limit)
<code>ds</code> , <code>margin</code>	float	actual spacing, and the margin used

`vehicle_margin(car_width, safety_buffer)`

```
return car_width / 2 + safety_buffer          # 0.30 / 2 + 0.05 = 0.20 m
```

The racing line is where the car's **centre** drives. The car's side is half its width away; the buffer covers tracking and localisation error. Rejects `car_width ≤ 0` or a negative buffer. Placeholders until the real car is measured.

`make_stations(track, spacing_target, margin, walls=None)`

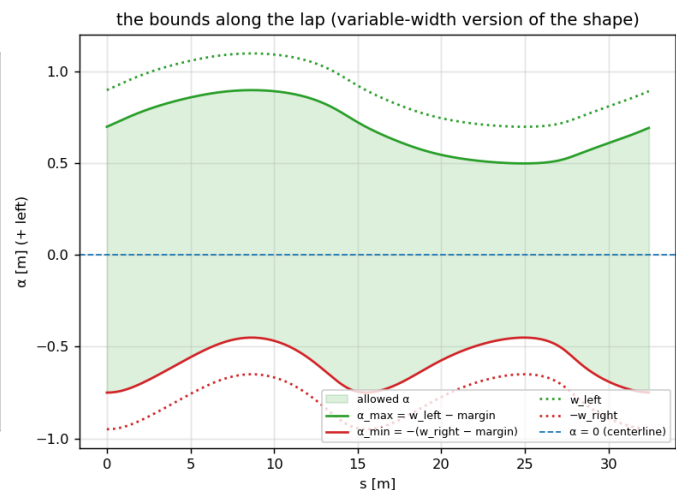
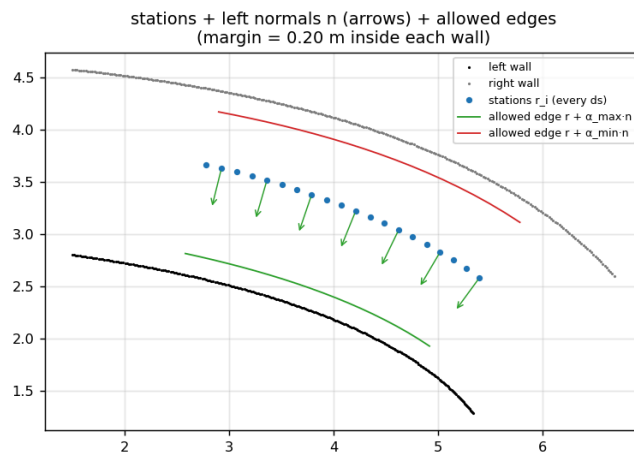
```

n = round(L / spacing_target)      # 32.558 / 0.15 = 217.05 → 217 stations
ds = L / n                        # 32.558 / 217 = 0.15004 m (exact: n · ds = L)
s = arange(n) * ds                # 0, ds, ..., L - ds
s, x, y, heading, kappa, w_left, w_right = track.sample(s)
normals = (-sin heading, cos heading)
if walls given: w_left, w_right = widths_from_walls(xy, normals, *walls) # inside the loop
alpha_max = w_left - margin        # furthest LEFT the centre may go
alpha_min = -(w_right - margin)    # furthest RIGHT (negative)

```

Why adjust ds: the curvature formula (section 4) assumes all neighbours are exactly ds apart — including the last station and the first. With $ds = 0.15$ exactly, the closing gap would be 0.058 m.

Check inside make_stations	Type	Catches
<code>alpha_max ≤ alpha_min</code> anywhere	error (ValueError)	track narrower than $2 \times \text{margin}$: no position fits. Message lists up to five <code>s</code> values.
$\alpha = 0$ outside the bounds (1 mm tolerance)	warning	a wall closer than the margin to the centerline. Only on the original track (<code>walls</code> is <code>None</code>): inside the loop the reference sits ON the margin by design.
<code>n < 3</code>	error	spacing larger than the track



1 stations (blue), left normals (arrows pointing to the left wall), and the allowed edges 0.20 m inside each wall. 2 the allowed band of α along the lap for a track whose widths vary.

build_walls(track, spacing=0.05) — the fixed walls

```

left_wall = r + w_left · n      # sampled every 5 cm, closed polylines
right_wall = r - w_right · n

```

Built **once** from the original centerline. They never move. Needed because once the reference is the racing line (not the centerline), its widths must be *measured*.

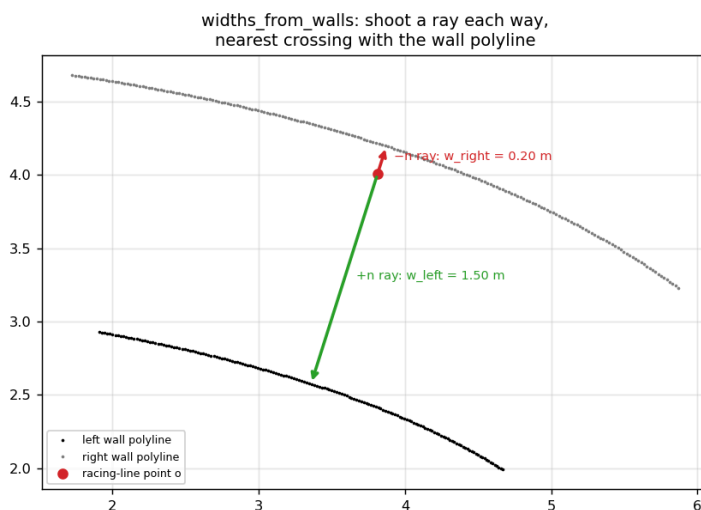
widths_from_walls — ray-casting

From each point `o`, shoot a ray along `+n` to the left wall and along `-n` to the right wall; the width is the distance to the **nearest** crossing. The maths (`_ray_distance`): ray `o + t·d` meets wall edge `a + u·e` (edge from `a` to the next wall point, `e = next - a`) when

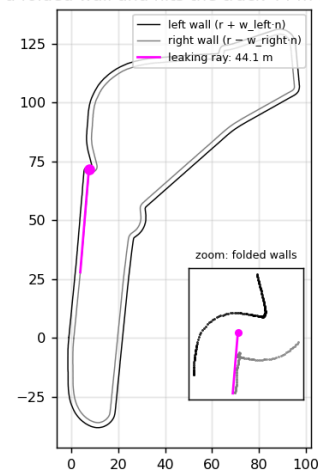
```
t·d - u·e = a - o
cross both sides with e:  t = ((a - o) × e) / (d × e)      (distance along the ray)
cross both sides with d:  u = ((a - o) × d) / (d × e)      (where on the edge, 0 ... 1)
a hit needs:  d × e ≠ 0 (not parallel),  t > 0 (in front),
              -1e-9 ≤ u ≤ 1 + 1e-9 (on the edge)
```

`×` is the 2-D cross product `a_x b_y - a_y b_x` (`_cross`). All rays `×` all edges are computed at once with numpy, in chunks of 256 rays so the arrays stay small ($256 \times 9,000$ instead of $3,000 \times 9,000$). A ray that hits nothing `→ ValueError` listing the indices.

Why the 1e-9 slack: stations (every 0.15 m) line up exactly with wall points (every 0.05 m), so a ray can pass exactly through a wall vertex: `u = 1` on one edge and `u = 0` on the next. Rounding made them 1.0000000001 and -0.0000000001, both rejected `→` "ray missed" on a perfect track. Found by a test.



Monza, round 2 at $s = 73.5$ m: the ray slips through a folded wall and hits the track 44 m away



1 the two rays from one racing-line point: 1.50 m to the left wall, 0.20 m (= margin) to the right wall. 2 what goes wrong on Monza: at the chicane the rebuilt wall folds over itself, a ray slips through the gap and hits the track 44 m away (section 7).

4. min_curvature.py — the optimisation

Input

Stations (one round) or a track model (the whole loop) + `kappa_max`, optional `lam`, OSQP settings

Output

QPResult (one round) / IterateResult (the loop): α , the racing points, predicted κ , cost, history

`kappa_max_from_steering(max_steer, wheelbase)`

Kinematic bicycle model: rear wheel straight, front wheel steered by δ ; both roll around one centre, radius $R = \text{wheelbase} / \tan \delta$. At full lock:

```
kappa_max = tan(max_steer) / wheelbase      # tan(0.4) / 0.33 = 1.281 1/m → R_min = 0.78 m
```

The QP must never ask for a tighter bend. Rejects `max_steer` outside $(0, \pi/2)$ or `wheelbase` ≤ 0 . Test: 45° and 0.5 m $\rightarrow \tan = 1 \rightarrow \kappa_{\max} = 2$.

Step B: `build_curvature_model(st) → (kappa_ref, M)`

Curvature at station i from its two neighbours: the sideways part of the "second difference" (how much the line bends between $i-1, i, i+1$):

```
 $\kappa_i \approx n_i \cdot (p_{i-1} - 2 p_i + p_{i+1}) / ds^2$       with  $p = r + \alpha n$ 
```

Put $p = r + \alpha n$ in, and everything splits into "reference part" + " α part":

```
kappa_ref_i = n_i \cdot (r_{i-1} - 2 r_i + r_{i+1}) / ds^2      (reference curvature,  $\alpha = 0$ )
M[i, i-1]   = (n_i \cdot n_{i-1}) / ds^2
M[i, i]     = -2 / ds^2 + 2 \cdot kappa_ref_i^2               ← correction, see below
M[i, i+1]   = (n_i \cdot n_{i+1}) / ds^2
→  $\kappa \approx \text{kappa\_ref} + M @ \alpha$       (LINEAR in  $\alpha$ )
```

Indices wrap (`(idx ± 1) % N`): station 0's left neighbour is station $N-1$. M has 3 non-zeros per row and is stored as a **sparse** matrix (only the non-zeros), built in one go from (value, row, column) lists.

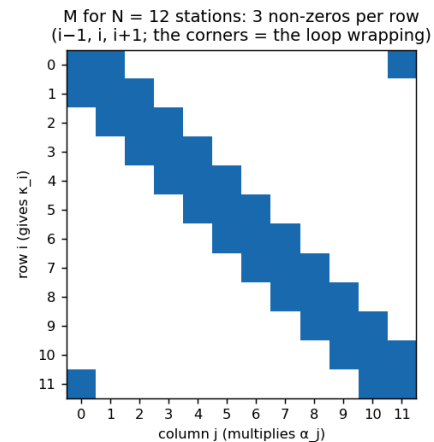
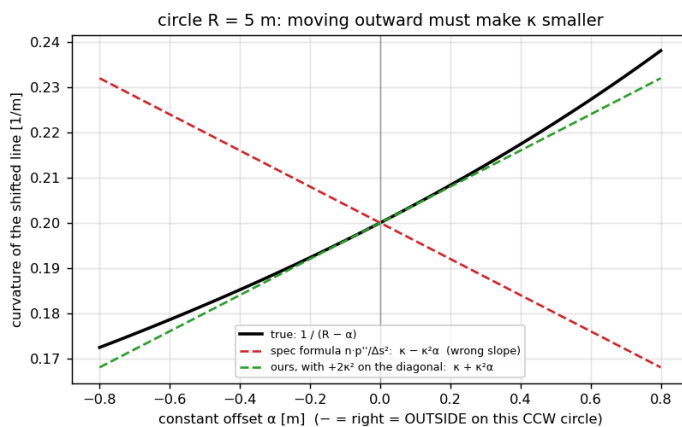
Correction — deviation from the spec (§5.2). The spec's formula (without $+2\kappa^2$) divides by the *reference* spacing ds , but a line moved outward is longer: its points are $(1 - \kappa\alpha)\cdot ds$ apart. Expanding the exact curvature of $p = r + \alpha n$ to first order:

exact: $\kappa_p = \kappa + \kappa^2 \alpha + \alpha''$
spec formula: $n \cdot p'' = \kappa - \kappa^2 \alpha + \alpha'' \rightarrow \text{missing } +2\kappa^2\alpha$

Checked numerically on a circle $R = 5$ (true curvature of the shifted circle = $1/(R - \alpha)$):

α (– = outside)	true	spec formula	with $+2\kappa^2$
–0.6	0.1786	0.2240 (wrong way!)	0.1760
0	0.2000	0.2000	0.2000
+0.6	0.2273	0.1760 (wrong way!)	0.2240

Without the fix the QP pushes the line to the **inside** wall and the spec's own circle test fails.



1 the three curvature formulas on a circle: only the corrected one follows the truth near $\alpha = 0$. 2 the shape of M for 12 stations: a band of 3 around the diagonal, plus two corners because the loop closes.

Step C: difference_matrix(N) and curvature_cost(kappa_ref, M, α , lam)

```
(D1 @  $\alpha$ )_i =  $\alpha_{i+1}$  -  $\alpha_i$  # closed loop
cost =  $\sum (\kappa_{ref} + M \alpha)^2 + \text{lam} \cdot \sum (\alpha_{i+1} - \alpha_i)^2$  # lam = 0 by default
```

The lam term is optional: it would punish sideways wiggles on long straights. Unused so far. At $\alpha = 0$ the cost is $\sum \kappa_{ref}^2$ (a test checks this): 15.70 for the T1 shape's centerline.

Step D: solve_min_curvature_qp(st, kappa_max, lam, osqp_settings)

OSQP solves problems written exactly like this ("standard form"):

$$\text{minimise} \quad \frac{1}{2} \alpha^T P \alpha + q^T \alpha \quad \text{subject to} \quad l_0 \leq A \alpha \leq h_1$$

Expanding our cost: $\|\kappa_{\text{ref}} + M\alpha\|^2 = \alpha^T (M^T M) \alpha + 2(M^T \kappa_{\text{ref}})^T \alpha + \text{const}$, so

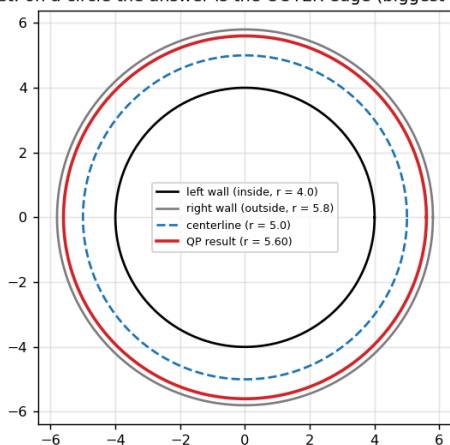
$$\begin{aligned} P &= 2 M^T M & (+ 2 \cdot \text{lam} \cdot D1^T D1) & & q &= 2 M^T \kappa_{\text{ref}} \\ A &= \begin{bmatrix} I \\ M \end{bmatrix} & l_0 &= \alpha_{\text{min}} & h_1 &= \alpha_{\text{max}} & (\text{stay inside}) \\ & & l_0 &= -\kappa_{\text{max}} - \kappa_{\text{ref}} & h_1 &= \kappa_{\text{max}} - \kappa_{\text{ref}} & (\text{if } \kappa_{\text{max}} \text{ given}) \end{aligned}$$

Scaling: M's entries are about $1/ds^2 \approx 44$, so P's are thousands. Everything is multiplied by ds^2 (the cost by ds^4): numbers around 1, easier for the solver. Multiplying a cost by a constant does not move its minimum. **Upper triangle:** P is symmetric, OSQP only reads the upper half (`sparse.triu`).

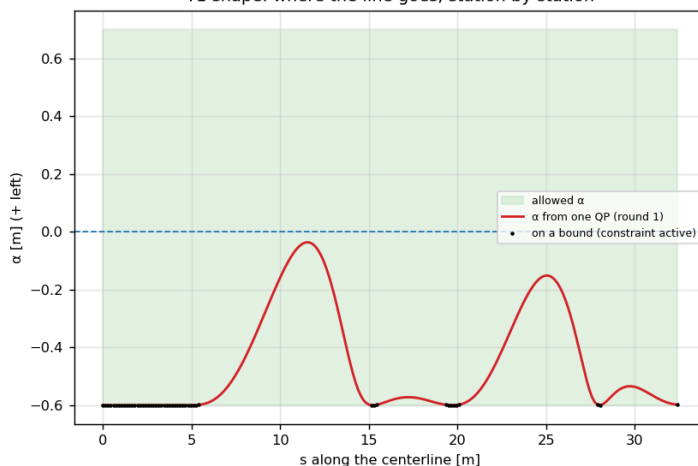
OSQP setting	Value	Why
<code>eps_abs</code> , <code>eps_rel</code>	1e-6	accuracy (spec)
<code>polishing</code>	True	final exact step on the active constraints (renamed from <code>polish</code> in osqp 1.x)
<code>max_iter</code>	100,000 (spec: 20,000)	Monza (2,975 stations) needs 59,675 iterations: the cost is a 4th-difference operator, badly conditioned. OSQP stops as soon as it has converged.

Any status other than "solved" (e.g. "solved inaccurate", "primal infeasible") → `RuntimeError` with the status. `QPResult`: `alpha`, `points = r + alpha n`, `kappa` (model prediction), `cost`, `status`, `solve_time`, `iterations`.

test: on a circle the answer is the OUTER edge (biggest radius)



T1 shape: where the line goes, station by station



1 the spec's test: constant-width circle → the line sits on the outer edge, $\alpha = -0.600$ m at every station (error $3 \cdot 10^{-16}$, 100 OSQP iterations). 2 T1 shape, round 1: α per station; black dots = on a bound (the car touches the margin there).

Step E: `iterate_min_curvature(track, spacing_target, margin, kappa_max, lam, max_iterations=5, convergence_tol=0.01, smooth_sigma=0)`

The linear model is exact only for $\alpha \rightarrow 0$. Round 1 moves the line up to 0.6 m, so its answer is approximate. Fix: use that answer as the new reference and solve again — now α is only the small *correction*, where the model is accurate.

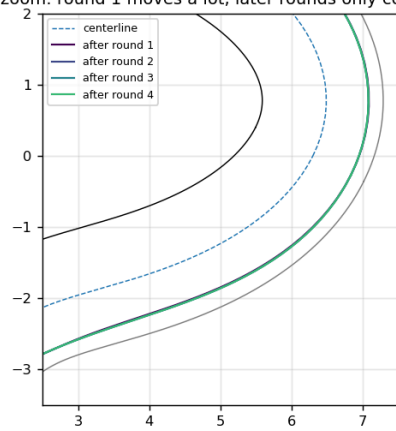
```
walls = build_walls(track)                # once, fixed
reference = track
for k in 1 ... max_iterations:
    st = make_stations(reference, ..., walls=None if k == 1 else walls) # A (+ ray-cast)
    qp = solve_min_curvature_qp(st, kappa_max, lam)                    # B-D
    log max| $\alpha$ |, peak | $\kappa$ |, cost, time, OSQP iterations
    reference = TrackModel(qp.points, widths=0, smooth_sigma)         # curve through p
    if max| $\alpha$ | < convergence_tol: converged; stop
    else: warning "did not converge in 5 rounds" (not an error)
```

The new reference's widths are zeros on purpose: the next round always re-measures them from the walls, and step F re-measures them after the last round.

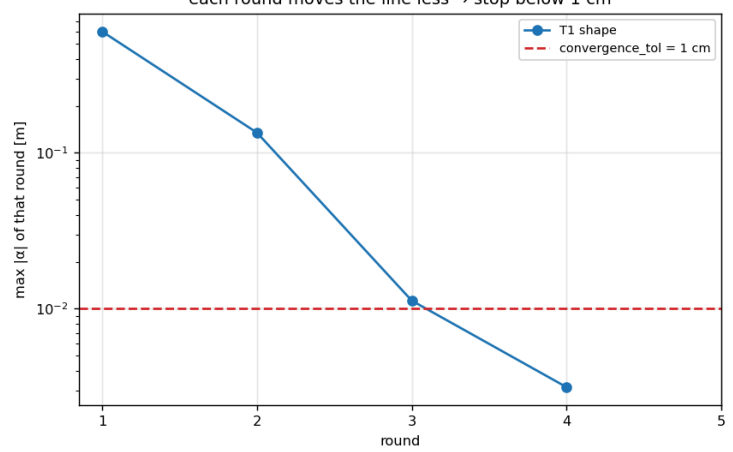
Round (T1 shape)	max α	peak κ (model)	cost (model)	OSQP iterations
1	0.600 m	0.385	8.77	1,250
2	0.135 m	0.434	11.39	4,750
3	0.011 m	0.433	11.39	850
4	0.003 m ✓	0.433	11.39	175

Why the cost goes UP after round 1: round 1's 8.77 is what the linear model *predicts* for a line 0.6 m away — too optimistic. From round 2 on the model is evaluated close to the line, so its numbers are real. Final 11.39 vs 15.70 for the centerline (−27%); peak κ 0.605 $\rightarrow$ 0.433.

zoom: round 1 moves a lot, later rounds only correct



each round moves the line less → stop below 1 cm



1 the line after each round: rounds 2–4 are nearly on top of round 1. 2 how far each round moved the line (log scale): stops at round 4, below 1 cm.

5. export.py — the final line and the files

Input

IterateResult + the original centerline track
model + final_spacing (0.10 m)

Output

RacingLine arrays → racing_line.csv +
racing_line_meta.json; load_racing_line →
TrackModel

The smooth curve through the last round's points already exists (`result.reference`). Missing: its widths (zeros), a proper `s = 0`, and the files.

`_start_on_line(line, centerline)` — where is `s = 0`?

```
r0, ψ0 = centerline at s = 0;  t0 = (cos ψ0, sin ψ0)      # driving direction at start
sample the racing line every 1 cm
ahead  = (p - r0) · t0          # signed distance PAST the start/finish line
beside = |(p - r0) × t0|        # distance ALONG the start/finish line
crossings: ahead goes from ≤ 0 to > 0
the start line also cuts the far side of the loop → keep the crossing nearest r0
linear interpolation inside the 1 cm step → exact s0
```

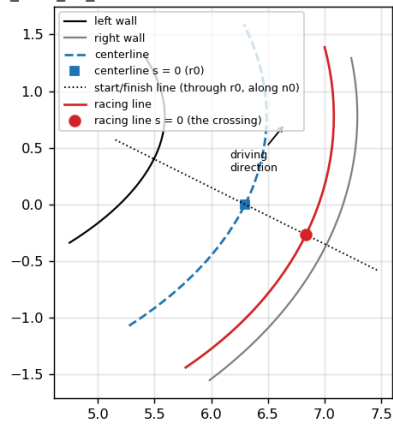
The start/finish line = the straight line through the centerline's first point, square to the driving direction. Both lines then start "abreast", like T1's own `s0`. Test: the racing line's first point is < 1 mm from that line.

`build_racing_line(result, centerline, final_spacing=0.10)`

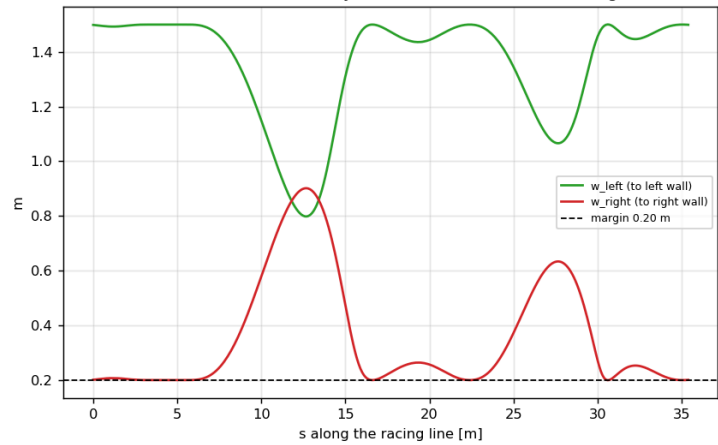
```
n = round(L / 0.10);  s = arange(n) · L/n                # 35.488 / 355 = 0.09997 m
x, y, heading, kappa = line.sample(s0 + s)               # sample() wraps modulo L
heading = where(heading ≤ -π, heading + 2π, heading)      # (-π, π] as the spec says
w_left, w_right = widths_from_walls(points, normals, *walls) # to the REAL walls
```

κ comes from the spline (smooth), not from the 3-point estimate. The widths are now asymmetric (e.g. 1.50 / 0.20 m) — correct: that is where the line sits.

_start_on_line: both lines share one start/finish line



re-measured widths: asymmetric, never below the margin



1 the shared start/finish line (dotted): the racing line's $s = 0$ is where it crosses it. 2 widths along the racing line: it touches the 0.20 m margin on the right (outside of the left loop) several times, never goes below.

Writing and reading

Function	What it does
<code>RacingLine.as_array()</code>	the 7 columns side by side, same order as the CSV
<code>build_meta(...)</code>	the JSON: method "min_curvature_qp" , time, map/frame (copied from T1's meta if given), SHA-256 of the source track.csv, margin, car width, buffer, $\kappa_{\max}$, both spacings, rounds run, converged, final $\max \alpha $
<code>write_racing_line(dir, line, meta)</code>	creates the folder, CSV with 6 decimals + JSON
<code>read_racing_line(csv)</code>	CSV $\rightarrow$ (M, 7) array
<code>load_racing_line(csv, meta)</code>	CSV $\rightarrow$ TrackModel (the racing line becomes the "centerline", widths = the corridor), meta attached as <code>.meta</code>

SHA-256 = a 64-character fingerprint of a file; one changed byte changes it completely $\rightarrow$ proves which track produced which racing line (same idea as T1's log hash).

6. The tests

Run: `python -m pytest tests/racing_line -q` → 27 passed (~5–15 s). Tests needing OSQP skip themselves if it is not installed.

Test file	Tests	What they prove
test_track_model_stub.py	5	$L = 2\pi R$; $\kappa = \pm 1/R$ with the right sign; s = real metres; closed/periodic; the normal points left
test_stations.py	7	uniform spacing, $N \cdot ds = L$, no station at L ; unit left normals; bounds values; narrow track → error with s ; ray-cast reproduces 0.9 / 0.8 m within 1 cm; ray miss → error; margin formula
test_min_curvature.py	9	$\kappa_{\text{ref}} \approx 1/R$ (1%); outward = flatter (the sign fix); model $\approx$ true 3-point curvature for small α ; cost at 0; circle → outer edge (1 mm); impossible κ limit → error; loop converges ≤ 5 rounds, each smaller, stays $\geq$ margin, peak κ lower; line hugs the outside + an apex exists; κ_{max} formula
test_export.py	6	circle → radius $R + 0.6$, $\kappa = 1/(R + 0.6)$; spacing ≈ 0.10 m, heading in $(-\pi, \pi]$; start < 1 mm from the start line; widths $\geq$ margin; write → load → sample gives the same numbers; two runs bit-identical

Using it from Python (there is no command line yet)

```
from t3_racing_line.track_model_stub import TrackModel
from t3_racing_line.stations import vehicle_margin
from t3_racing_line.min_curvature import iterate_min_curvature, kappa_max_from_steering
from t3_racing_line.export import build_racing_line, build_meta, write_racing_line

track = TrackModel.from_csv("out/track/track.csv")
margin = vehicle_margin(car_width=0.30, safety_buffer=0.05)
kmax = kappa_max_from_steering(max_steer=0.4, wheelbase=0.33)
result = iterate_min_curvature(track, 0.15, margin, kappa_max=kmax)
line = build_racing_line(result, track, final_spacing=0.10)
meta = build_meta(line, result, kappa_max=kmax, final_spacing=0.10,
                  car_width=0.30, safety_buffer=0.05)
write_racing_line("out/racing_line", line, meta)
```

7. How it was tested — results and lessons

Track	Result
circle $R = 5$, widths 1.0 / 0.8	one QP: $\alpha = -0.600$ m everywhere (outer edge), exact
stadium (two 10 m straights, $R = 3$ turns)	3 rounds: $0.80 \rightarrow 0.068 \rightarrow 0.0024$ m. No inside apex: with parallel straights the widest 180° turn is simply the outer edge.
T1 synthetic shape	4 rounds ($0.600 \rightarrow 0.003$ m); peak κ $0.605 \rightarrow 0.433$; cost $15.70 \rightarrow 11.39$; line $32.56 \rightarrow 35.49$ m long (it drives wider)
Monza (T1 output from the F1TENTH gym), one QP	2,975 stations; peak κ $1.419 \rightarrow 0.625$; 59,675 OSQP iterations, ~8–12 s
Monza, full loop	does not converge : round 2 ray leaks 44 m at $s = 73.5$ m (chicane). Left until the real T2.

Lessons found by testing

- **The spec's curvature formula had the wrong sign on the stretching term** → the QP would hug the inside wall. Fixed with $+2\kappa^2$ on M's diagonal (section 4).
- **Big tracks need many solver iterations**: 20,000 was not enough for Monza → 100,000.
- **Rays through a wall vertex** were rejected by rounding → $1e-9$ slack.
- **"Outside–apex–outside" is not always the answer**: on a circle, a stadium or one big loop the best line hugs the outside; the apex appears only in bends that turn against the loop.
- **Noisy curvature folds the walls** (inside width $> 1/|\kappa|$): rays leak across the infield. T1 already warns about this ("wall folds", Monza: 13 stations with $|\kappa| \cdot w \geq 1$, worst 1.54). Needs T2's smoothing; T3 should later fail loudly on an impossible width.

8. Deviations from the spec and open items

Spec says	We do	Why
§5.2 $M[i,i] = -2/ds^2$	$-2/ds^2 + 2\kappa_{ref}^2$	otherwise moving outward looks more curved (section 4)
§4 OSQP max_iter 20,000	100,000	Monza needs ~60,000
§10 step 5: oval → outside–apex–outside	test "hugs outside + an apex exists"	on the test shapes the true optimum hugs the outside
§12 many small files	4 files grouped by stage	team preference

Not built yet: §7 checks as a report (inside-corner rule, $|\kappa| \leq \kappa_{max}$, seam, spikes), shortest-path baseline + rough lap-time comparison, a config file, a command-line script, the TUM benchmark.

Waiting for: the real T2 (swap the stub, re-run Monza) and the control team's answers on the rulebook (centerline-indexed export, shared track file, vehicle config with wheelbase + width, margin, lap direction).

9. Glossary

Word	Meaning
racing line	the path the car's centre should follow
station	one of the evenly spaced points (every ~0.15 m) where the line may move sideways
α (alpha), lateral offset	how far the line moves sideways at a station, along the left normal: + left, – right (same as the controller's e_y)
normal n	unit arrow at 90° to the LEFT of the driving direction: $(-\sin \psi, \cos \psi)$
heading ψ	direction of travel, radians, $0 = +x$ axis, counter-clockwise positive
curvature κ	how sharply the line bends = $1 / \text{radius}$; + left turn, – right turn
arc length s vs chord	distance along the curve vs the straight line between two points (always shorter)
spline (cubic, periodic)	smooth curve through points made of cubic pieces; periodic = end joins start smoothly
margin	closest the car's centre may get to a wall = half the car + buffer (0.20 m)
κ_{max}	tightest bend the steering allows = $\tan(\text{max_steer}) / \text{wheelbase}$ (1.28 1/m)
linear / linearise	approximate by a straight-line relation (here $\kappa \approx \kappa_{\text{ref}} + M\alpha$), exact only for small α
QP (quadratic program)	minimise a squared cost under linear limits; has one best answer, solved exactly
OSQP	the QP solver library we use (iterative: improves the answer step by step until accurate)
active constraint / on a bound	a station where α sits at α_{min} or α_{max} : the car touches the margin there
apex	where the line touches the INSIDE of a bend
round / iteration	one pass of stations $\rightarrow$ model $\rightarrow$ QP $\rightarrow$ new reference
sparse matrix	stores only the non-zero entries (M : 3 per row instead of N)
ray-casting	shooting a straight ray from a point and finding where it first hits a polyline
fold (wall fold)	a wall rebuilt as line $\pm$ width looping over itself where the bend radius is smaller than the width
Frenet coordinates (s , e_y)	"distance along the centerline, sideways offset from it" — what the controller's rulebook uses